

蚁群算法和粒子群算法

蚁群算法 (ASO - Ant System Optimization)

- 基本原理：蚁群算法是一种模拟蚂蚁觅食的原理设计出的一种群集智能算法，蚂蚁对应一个计算智能体，依概率选择移动方向，在经过的路径上留下信息素，信息素随时间挥发，信息素浓度大被选的概率更大
- 算法过程：

```
(1) 初始化 随机放置蚂蚁
(2) 迭代 iter=1
while iter <= ItCount do (执行迭代) //ItCount是预设的迭代次数
    for i=1 to m do (对m只蚂蚁循环)
        for j=1 to n-1 do (对n个城市循环)
            采用轮盘赌方法选择下一个城市j;
            将城市j放入禁忌表;
            蚂蚁转移到城市j;
        end_for
    end_for

    计算每只蚂蚁经过的路径长度
    更新所有蚂蚁路径上的信息量
    iter = iter + 1;
end_while
(3) 输出结果结束算法
```

- 适用范围：适用于在图上寻找最优路径，适用于离散问题最优解。

粒子群算法 (PSO - Particle Swarm Optimization)

- 基本原理：
是一种基于如蚁群寻优的启发式搜索算法，一种随机优化方法，通过粒子群在解空间中进行搜索寻找最优解。
- 算法流程：

粒子历史最优位置 p_best^i 全局最优历史最优位置 g_best

■ 算法过程描述

□ 初始化

- 初始化粒子群：每个粒子的位置和速度，即 $x_0^{(i)}$ 和 $v_0^{(i)}$
- $p_{best}^{(i)}$ 和 g_{best}

□ 循环执行如下三步直至满足结束条件

- 计算每个粒子的适应度： $f(x_n^{(i)})$
- 更新每个粒子历史最好适应度及其相应的位置，更新当前全局最好适应度及其相应的位置
- 更新每个粒子的速度和位置

$$v_{n+1}^{(i)} = v_n^{(i)} + c_1 * r_1 * (p_{best}^{(i)} - x_n^{(i)}) + c_2 * r_2 * (g_{best} - x_n^{(i)})$$

$$x_{n+1}^{(i)} = x_n^{(i)} + v_{n+1}^{(i)}$$

<https://blog.csdn.net/yuanmingsun>

//i是第i个粒子 n是迭代次数

//f是目标 f () 是适应度

//ci是学习因子，控制粒子向历史最优位置移动的速度，ci越大，粒子向历史最优位置移动的速度越快，ri是从区间 [0,1] 上均匀分布的随机数，引入了随机性，使粒子不会每次都以相同的方式更新速度，增强搜索的多样性。

- 适用范围：适用于求解连续解空间的优化问题。

bp必考

反向传播算法 (Backpropagation, BP) :

- 基本原理:

反向传播算法是一种用于训练人工神经网络的算法，通过计算输出层与目标值之间的误差，利用链式法则 (Chain Rule) 从输出层向输入层逐层计算损失函数对各权重的梯度，反向传播误差到隐藏层，再用梯度下降法更新网络权重，以最小化误差。

- 算法流程:

1.前向传播：输入样本，通过神经网络计算输出。

2.计算误差：将输出与目标值进行比较，计算误差。

$$\text{计算损失 } E = \frac{1}{2} \|y - d\|^2 \quad (d \text{ 为真实标签})。$$

3.反向传播误差：从输出层开始，逐层计算误差梯度，反向传播到隐藏层。（这里的逐层计算公式比较复杂）

4.更新权重：根据误差对权重的偏导（梯度）和学习率，更新神经网络的权重。

步骤 3: 参数更新 (Gradient Descent)

使用梯度下降更新权重和偏置:

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial E}{\partial W^{(l)}}, \quad b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial E}{\partial b^{(l)}}$$

其中 η 是学习率。

5. 迭代训练: 重复前向传播、计算误差、反向传播误差和更新权重, 直到满足停止条件 (如达到最大迭代次数或误差足够小)。

利用链式法则 (Chain Rule) 从输出层向输入层逐层计算损失函数对各权重的梯度, 再用梯度下降法更新权重。

可能会发生的问题:

1. 梯度消失问题: 在反向传播过程中, 当激活函数的导数接近于零, 梯度就会变小, 导致权重更新缓慢, 甚至无法更新。(sigmoid函数的特性导致的)
2. 梯度爆炸问题: 当神经网络的层数较多时, 反向传播过程中计算的梯度可能会变得非常大, 导致权重更新异常, 甚至训练失败。
3. 局部最优解问题: 由于反向传播算法是基于梯度下降的, 所以可能会陷入局部最优解, 而不是全局最优解。

三个问题的缓解方法:

1. 梯度消失问题: 使用ReLU等激活函数, 而不是sigmoid函数。
2. 梯度爆炸问题: 使用梯度裁剪 (Gradient Clipping) 技术, 限制梯度的最大范数。
3. 局部最优解问题: 使用momentum (动量) 项, 加速梯度下降过程, 避免陷入局部最优解; 或者使用自适应学习率 (如Adam优化器), 根据梯度的变化动态调整学习率。

格子游戏问题(Gridworld) 强化学习 贝尔曼方程

强化学习的贝尔曼方程:

贝尔曼方程是强化学习中的一个重要公式, 用于计算状态值函数 (state-value function) 或动作值函数 (action-value function)。它描述了在每个状态下, 智能体应该选择的最优动作的价值。

策略和状态估值函数

■ 策略

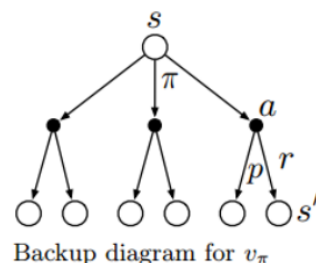
- 状态到行为的映射
- 随机式策略： $\pi(a|s)$
- 确定式策略： $a = \pi(s)$

■ 给定策略 π ，状态估值函数(state-value function) 定义为

$$v_{\pi}(s) \doteq \mathbb{E}_{\pi}[G_t \mid S_t=s] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t=s\right], \text{ for all } s \in \mathcal{S}$$

贝尔曼方程

■ 状态估值函数的贝尔曼方程



$$\begin{aligned} v_{\pi}(s) &\doteq \mathbb{E}_{\pi}[G_t \mid S_t=s] \\ &= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t=s] \\ &= \sum_a \pi(a|s) \sum_{s'} \sum_r p(s', r | s, a) \left[r + \gamma \mathbb{E}_{\pi}[G_{t+1} | S_{t+1}=s'] \right] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r | s, a) \left[r + \gamma v_{\pi}(s') \right], \quad \text{for all } s \in \mathcal{S} \end{aligned}$$

Tip!

Bellman equation expresses a relationship between the value of a state and the values of its successor states

策略估值：

策略估值是指在强化学习中，根据当前状态和策略，估计该状态下执行该策略的价值。它是贝尔曼方程的一个特殊情况，用于计算动作值函数（action-value function）。

策略估值

- 给定策略 π ，该策略下的状态估值函数满足

$$\begin{aligned} v_{\pi}(s) &\doteq \mathbb{E}_{\pi}[G_t \mid S_t = s] \\ &= \mathbb{E}_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \mathbb{E}_{\pi}[R_{t+1} + \gamma v_{\pi}(S_{t+1}) \mid S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma v_{\pi}(s')] \end{aligned}$$

- 假如环境 $p(s', r|s, a)$ 已知，状态估值函数的求解方式有
 - 求解线性方程组
 - 计算开销大
 - 寻找不动点——迭代策略估值
-

策略提升

■ 策略估值的目的是为了寻找更优的策略（策略提升）

- 策略估值根据策略 π 计算其估值函数 v_π

■ 策略提升

- 根据当前策略的估值函数，寻找更优的策略（如果存在），逐步找到最优策略

- 根据策略 π 的估值函数 v 寻找更优策略 π'

- 提升方法

- 给定一个确定策略 π ，在状态 s 下选择行为 a ，后续按照策略 π 行动所得的估值 $q_\pi(s, a)$ 是否高于完全按照策略 π 行动得到的估值 $v_\pi(s)$

$$\begin{aligned} q_\pi(s, a) &\doteq \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')] \end{aligned}$$

策略迭代

- 从初始策略 π_0 开始，迭代进行“策略估值(E)”和“策略提升(I)”，最终得到最优策略 π_*

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*$$

Policy iteration (using iterative policy evaluation)

1. Initialization
 $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$
2. Policy Evaluation
Repeat
 $\Delta \leftarrow 0$
 For each $s \in \mathcal{S}$:
 $v \leftarrow V(s)$
 $V(s) \leftarrow \sum_{s', r} p(s', r \mid s, \pi(s)) [r + \gamma V(s')]$
 $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
 until $\Delta < \theta$ (a small positive number)
3. Policy Improvement
 $\text{policy-stable} \leftarrow \text{true}$
 For each $s \in \mathcal{S}$:
 $\text{old-action} \leftarrow \pi(s)$
 $\pi(s) \leftarrow \arg\max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma V(s')]$
 If $\text{old-action} \neq \pi(s)$, then $\text{policy-stable} \leftarrow \text{false}$
 If policy-stable , then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

例子：

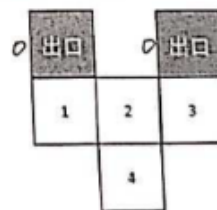
3. 格子游戏：假如玩家采用的策略是在每个格子以等概率选择上下左右四个方向进行移动，每移动一步的收益是-1，移动到出口时游戏结束。如果当前移动会导致出界，那么移动后位置不变。

(1) 策略评估：请使用动态规划方法计算出当前策略下每个格子对应的状态估值。（8分）

(2) 策略提升：写出上述估值函数对应的贪心策略。（4分）

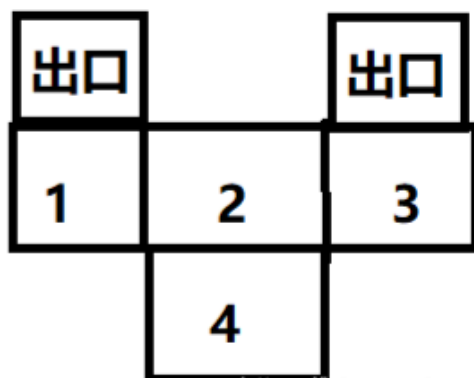
(3) 最优策略：求解该问题的最优策略及相应的状态估值。（8分）

提示：动态规划求解可以通过列出方程组进行求解。



题目描述-格子游戏：

每一个格子等概率向着4个方向移动，每次移动一步，收益为 -1，移动到出口结束游戏。若当前移动会导致出界，则移动后位置不变：



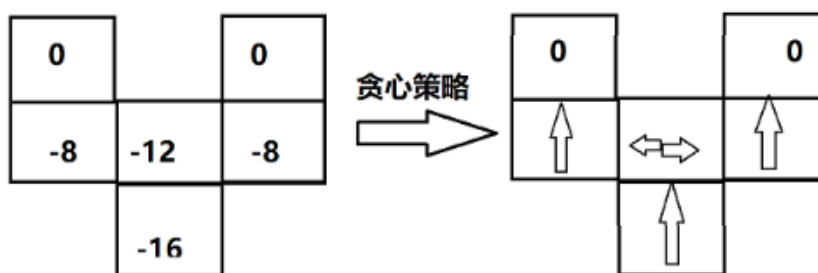
(1) 策略估值：使用动态规划方法求当前策略下每一格子对应的状态估值

解：

$$\begin{cases} V_1 = 1/4 (0 + V_1 + V_1 + V_2) - 1 \\ V_2 = 1/4 (V_2 + V_4 + V_1 + V_3) - 1 \\ V_3 = 1/4 (0 + V_3 + V_2 + V_3) - 1 \\ V_4 = 1/4 (V_2 + V_4 + V_4 + V_4) - 1 \end{cases} \Rightarrow \begin{cases} V_1 = -8 \\ V_2 = -12 \\ V_3 = -8 \\ V_4 = -16 \end{cases}$$

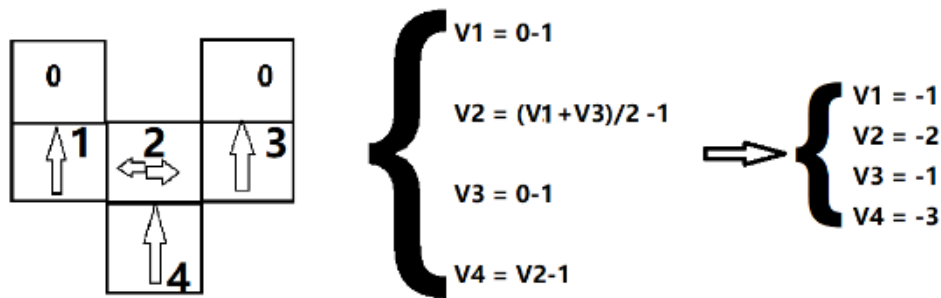
(2) 策略提升：写出上述估值函数对应的贪心策略

解：

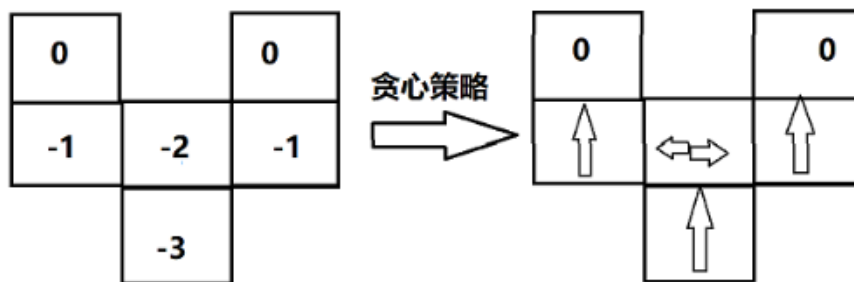


(3) 最优策略：求解该问题最优策略及其相应的状态估值

解：



状态不变，最大策略为：



- 看了例子就好理解了

蒙特卡洛方法：

蒙特卡洛方法是一种基于随机抽样的统计方法，用于估计未知量的数值。它通过模拟随机事件的发生次数来获得近似解。

(抽一万次牌推测牌库)

A*搜索算法 启发式路径算法 为什么树图 可采纳 一致 和最优路径找不找得到

- A*搜索算法是一种启发式路径搜索算法
公式： $f(x) = g(x) + h(x)$
- $g(x)$ 是从起始节点到节点x的实际代价
- $h(x)$ 是节点x到目标节点的启发式估计代价
- 可采纳含义：启发函数 $h(x)$ 永远不会高估从x到目标节点的真实最小代价。
如果 $h(n)$ 可采纳，且使用图搜索，则A一定能找到最优路径；
- 更强的条件：一致性：启发函数 $h(x)$ 是一致的，即 $h(x) \leq c(x, y) + h(y)$ ，其中 $c(x, y)$ 是从x到y的实际代价。
如果一致，必定可采纳，反之不一定
- A*搜索树的最优条件:启发函数 $h(x)$ 是可采纳的
证明：

A*搜索算法和其证明 吴老师部分搜索

A*图搜索的最优性条件是什么? 启发函数一致性

证明: 如果启发函数 $h(n)$ 可采纳, 那么树搜索的A*算法是最优的。*2证明

可采纳性的意义:

$$h(A) \leq \text{actual cost from A to G}$$

一致性的意义:

$$h(A) - h(C) \leq \text{cost}(A \text{ to } C)$$

3.11-一致性.

(1) 假设最优解 x^* , A*算法的解不是最优, 则:
 n 是 x^* 任意祖先.

$$x^* \text{ 最优性} \Rightarrow g(x^*) < g(x)$$

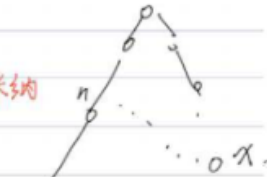
$$\text{又: } h(x^*) = 0, h(x) \geq 0. \quad h \text{ 可采纳}$$

$$\therefore f(x^*) < f(x)$$

$$\begin{aligned} h \text{ 可采纳} \Rightarrow f(n) &= g(n) + h(n) \leq g(n) + h^*(n) \\ &= g(x^*) = g(x^*) + h(x^*) = f(x^*). \end{aligned}$$

$$\therefore f(n) \leq f(x^*) < f(x)$$

$\therefore x^*$ 任意祖先都应先于 x 被扩展, $\therefore x^*$ 比 x 先被扩展.
矛盾, $\therefore$ A*算法能找到最优解.



- A*搜索图的最优条件: 启发函数 $h(x)$ 是一致的
证明:

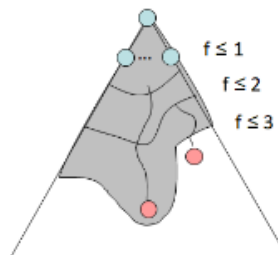
A*图搜索最优的条件是启发式函数 $h(n)$ 是一致的，证明如下：

A*算法采用一致的启发函数：

Fact 1: A*算法扩展节点， f 值单调增
(f -contours)

Fact 2: 对每个状态 s , 到达 s 最优的节点先于
次优的节点扩展

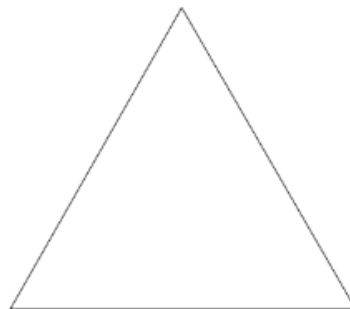
Result: A*图搜索是最优的



<https://blog.csdn.net/yuanninesuns>

证明:

- 假定 到 G^* 的路径上某个 n 不能进入队列中，
因为某个具有相同状态且较差的 n' 先被扩展了 (disaster!)
- 找到树中最高的这样的节点 n
- 设 p 是 n 的祖先，且在 n' 出列时在队列中
- $f(p) < f(n)$ consistency
- $f(n) < f(n')$, n' 次优
- p 应该在 n' 之前被扩展
- 矛盾!



<https://blog.csdn.net/yuanninesuns>

表 4.3: 不同搜索框架下 A* 搜索的最优性变化		
	可采纳的启发式函数	一致的启发式函数
树搜索	一定找到最优路径	一定找到最优路径
图搜索	不保证找到最优路径	一定找到最优路径

感知机（高级ai的感知机不深）

- 感知机以超平面来划分两类样本；对于线性可分的训练集，感知机算法在有限步内收敛；感知器学习是在假设空间中选取使损失函数最小的模型参数
- $wx+b=0$ 是超平面，将样本空间分成了两部分，一部分是 $wx+b>0$ ，另一部分是 $wx+b<0$ 。
用有标注的样本带进去训练（方程组），就行了

归结原理（resolution） 假言推理（modus ponens） 必考

$KB \models \alpha$ if and only if $(KB \Rightarrow \alpha)$ is valid.

蕴含

$KB \models a$, KB蕴含a,KB是小圈，被包在a的大圈里

(1)

- 知识表示：将人类知识用计算机可处理的形式（如逻辑、规则）表达出来。
- 自动化知识推理：计算机基于已有知识，自动推导新知识或结论的过程。
- Prolog：既是知识表示（写事实/规则），也是知识推理（回答查询）。

(2)

设 $M(\phi)$ 为公式 ϕ 的模型集合，则：

$M(\phi \wedge \psi) = M(\phi) \cap M(\psi), \quad M(\phi \vee \psi) = M(\phi) \cup M(\psi)$

(3)

- $\models$ ：语义蕴含，表示“从前提可推出结论”（在所有模型中成立）。
- $\Rightarrow$ ：逻辑连接词，表示“如果.....那么.....”，是句子的一部分。

表格		
符号	名称	用途
$\models$ (Turnstile / Semantic Entailment)	逻辑蕴含 (语义蕴含)	表示：在所有模型中，若前提为真，则结论也为真。 例如： $\Gamma \models \phi$ 表示：只要 Γ 中所有公式为真，则 ϕ 必然为真。 这是语义层面的关系，不依赖具体推理规则。
$\Rightarrow$ (Implication)	逻辑连接词 (蕴涵)	是命题逻辑中的语法符号，表示一个复合句子：“如果 A，则 B”。 例如： $A \Rightarrow B$ 是一个逻辑公式，其真值由真值表决定（只有当 A 真 B 假时为假）。 它是句子内部的构造，不是推理关系。

CNF：合取范式(归结原理应用)

Conversion to CNF

- $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
- 1. Eliminate $\Leftrightarrow$, replacing $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$
 $(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
- 2. Eliminate $\Rightarrow$, replacing $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$.
 $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$
- 3. Move $\neg$ inwards using de Morgan's rules and double-negation
 $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$
- 4. Apply distributivity law ($\vee$ over $\wedge$) and flatten:
 $(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$



(3) 将 $S_1 \sim S_6$ 转化为合取范式:

$$S_1: A \Leftrightarrow (B \vee E), \quad \textcircled{1} \quad \textcircled{2} \quad \textcircled{3}$$

$$S_2: E \Rightarrow D, \quad \textcircled{4} \quad \textcircled{5}$$

$$S_3: (C \wedge F) \Rightarrow \neg B, \quad \textcircled{6} \quad \textcircled{7} \quad \textcircled{8}$$

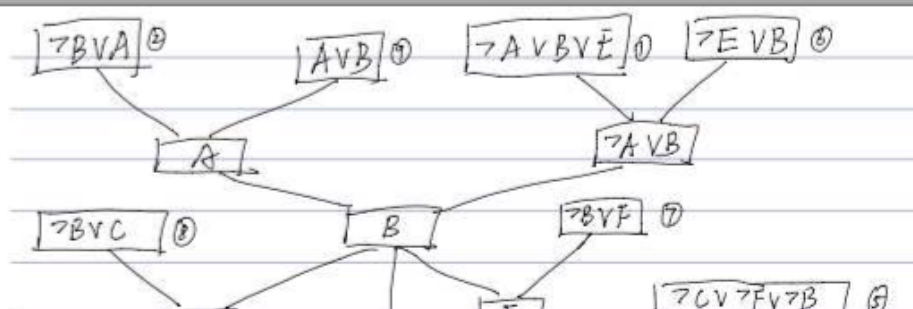
$$S_4: E \Rightarrow B, \quad \textcircled{9} \quad \textcircled{10}$$

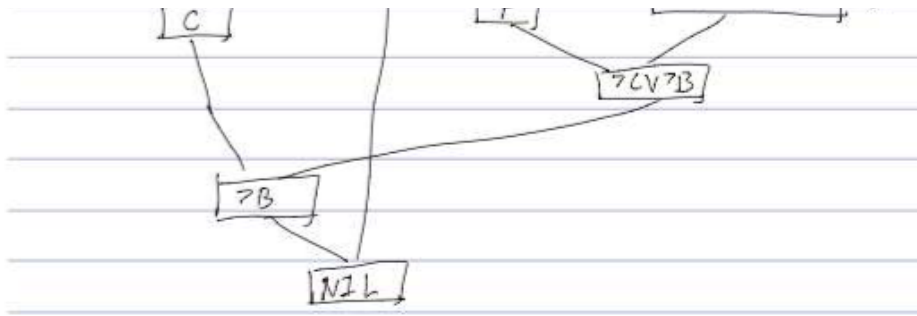
$$S_5: B \Rightarrow F, \quad \textcircled{11} \quad \textcircled{12}$$

$$S_6: B \Rightarrow C, \quad \textcircled{13} \quad \textcircled{14}$$

结论取反 $\textcircled{15} \neg A \vee \neg B$

初始时有 $\textcircled{1} \sim \textcircled{15}$ 九个子句。





逻辑连接词真值表

表 2.2: 逻辑连接词的真值表

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
False	False	True	False	False	True	True
False	True	True	False	True	True	False
True	False	False	False	True	False	False
True	True	False	True	True	True	True

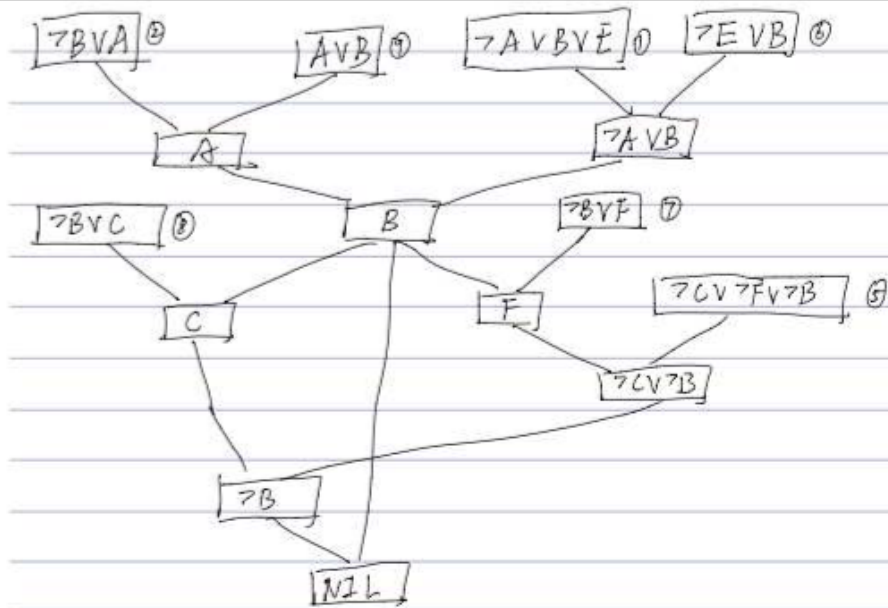
归结原理

(3) 将 $s_1 \sim s_6$ 转化为合取范式.

$$S_1: A \Leftrightarrow (B \vee E), \quad \overset{10}{(\neg A \vee B \vee E)} \wedge \overset{2}{(\neg B \vee A)} \wedge \overset{3}{(\neg E \vee A)}$$
$$S_2: E \Rightarrow D, \quad \textcircled{9} \quad \underline{E \vee D}$$
$$S_3: (C \wedge F) \Rightarrow \neg B, \quad \neg C \vee \neg F \vee \neg B.$$
$$S_4: E \Rightarrow B, \quad (16) \quad \underline{\neg E \vee B}.$$
$$\text{S5: } B \Rightarrow F, \quad \textcircled{1} \supset \underline{B \vee F}$$
$$\text{So: } B \Rightarrow C, \text{ (8) } \neg B \vee C$$

结论取反 (9) $A \vee B$

初始时有 ①~⑨ 九个句子.



- 可靠性证明 (soundness) :

一阶谓词逻辑*2

2. (1) 请使用语义网络的方式表达如下事实: (3分)

Bernardo Huberman is a Senior HP Fellow and Director of the Social Computing Lab at Hewlett-Packard Laboratories. He received his Ph.D. in Physics from the University of Pennsylvania, and is currently a Consulting Professor in the Department of Applied Physics at Stanford University.

(2) 请用一阶谓词逻辑表达如下语句: 胜者为王, 败者为寇: (3分)

(3) 请用模糊逻辑表达句子: 很少有成绩好的学生特别贪玩。注意刻画其中的模糊量词、模糊修饰词等。(4分)

胜者为王败者为寇

$$\begin{aligned} (2) \quad & \forall x (Person(x) \wedge Winner(x) \Rightarrow King(x)) \\ & \forall x (Person(x) \wedge Loser(x) \Rightarrow Coward(x)) \end{aligned}$$

不到长城非好汉

模糊逻辑表达*2

请用模糊逻辑表达句子: 很少有成绩好的学生特别贪玩。注意刻画模糊量词, 和模糊修饰词。

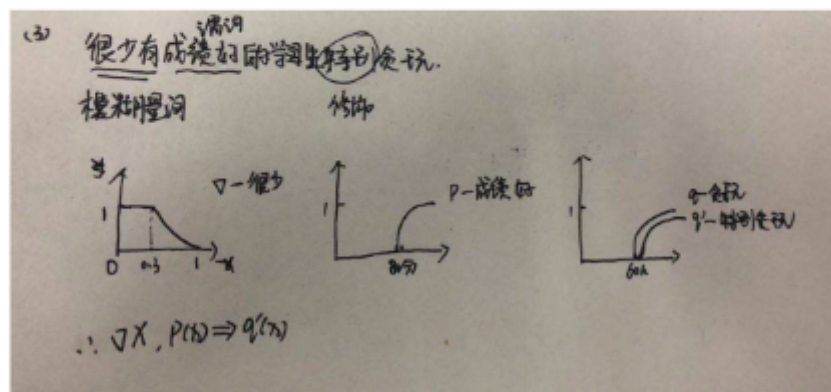
大多数成绩好的学生学习都很刻苦。
很少有成绩好的学生特别贪玩。

分别刻画模糊谓词、模糊修饰词、模糊量词

模糊谓词: 成绩好、刻苦、贪玩

模糊修饰词: 都很、特别

模糊量词: 大多数、很少



// li 和 mj 是互补子句 (complementary literals), 即 li 和 mj 中至少有一个为假。

- 完备性证明 (completeness):

要证明:

如果一个逻辑公式永假, 那么一定可以通过归结法推导出空子句 ($\perp$)。

证明思路

反证: 如果归结法怎么都推不出空子句 ($\perp$), 那这个公式一定是可满足的 (有办法让它为真)

1. 假设对 S 进行所有可能的归结，得到闭包 S^* ，且 $\square \notin S^*$ （即推不出空子句）。
2. 构造一个真值指派 M ：
对每个原子 P ，令
$$M(P) = \begin{cases} \text{false}, & \text{若 } \neg P \in S^* \\ \text{true}, & \text{否则} \end{cases}$$
3. 关键性质：由于 $\square \notin S^*$ ，不可能同时有 $P \in S^*$ 和 $\neg P \in S^*$ （否则可归结出 $\square$ ）。所以 M 是良定义的。
4. 验证 $M \models S$ ：
任取子句 $C \in S \subseteq S^*$ 。
 - 若 C 全为负文字，比如 $\neg P_1 \vee \dots \vee \neg P_k$ ，则每个 $\neg P_i \in S^*$ ，故 $M(P_i) = \text{false}$ ，所以 $\neg P_i = \text{true}$ ， C 为真。
 - 若 C 含正文字 P ，而 $M(P) = \text{false}$ ，说明 $\neg P \in S^*$ 。但此时可用 C 与 $\neg P$ 归结，消去 P ，得到更短子句，最终会推出 $\square$ （因 C 所有文字都假 $\Rightarrow$ 可逐步归结为空）。这与假设矛盾。
 $\rightarrow$ 故 C 中必有文字在 M 下为真。
5. 因此 $M \models S$ ，即 S 可满足。
6. 由逆否命题得：若 S 不可满足，则必能归结出 $\square$ 。✔

假言推理 (modus ponens)

原理：

用逻辑表达式可写为：

$$(P \rightarrow Q) \wedge P \Rightarrow Q$$

命题逻辑和一阶谓词逻辑

- 命题逻辑：研究命题之间的逻辑关系和推理规则的形式系统。命题是可以判断真假的陈述句，如“今天下雨了”。
- 一阶谓词逻辑：在命题逻辑的基础上引入了变量和量词，用于描述更复杂的关系和推理。例如，“所有的人都是人” ($\forall x \text{ Person}(x)$)。
- 例子：

一阶谓词逻辑*2

2. (1) 请使用语义网络的方式表达如下事实: (3分)

Bernardo Huberman is a Senior HP Fellow and Director of the Social Computing Lab at Hewlett-Packard Laboratories. He received his Ph.D. in Physics from the University of Pennsylvania, and is currently a Consulting Professor in the Department of Applied Physics at Stanford University.

(2) 请用一阶谓词逻辑表达如下语句: 胜者为王, 败者为寇: (3分)

(3) 请用模糊逻辑表达句子: 很少有成绩好的学生特别贪玩, 注意刻画其中的模糊量词、模糊修饰词等。 (4分)

胜者为王败者为寇

$$\begin{aligned} (2) \quad & \forall x (Person(x) \wedge Winner(x) \Rightarrow King(x)) \\ & \forall x (Person(x) \wedge Loser(x) \Rightarrow Coward(x)) \end{aligned}$$

不到长城非好汉

模糊逻辑表达*2

请用模糊逻辑表达句子, 很少有成绩好的学生特别贪玩。注意刻画模糊量词, 和模糊修饰词。

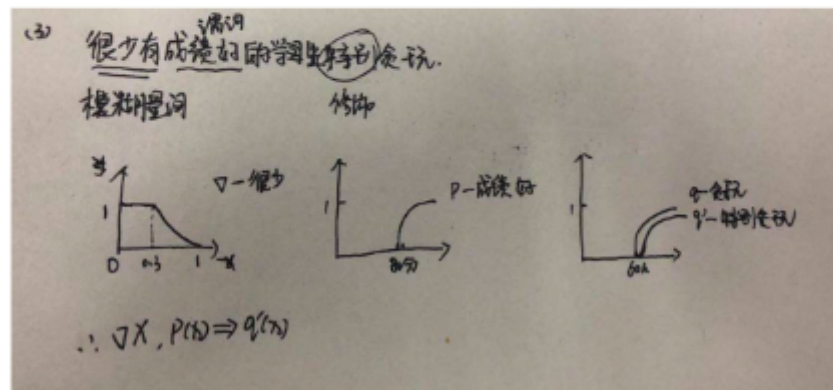
大多数成绩好的学生学习都很刻苦。
很少有成绩好的学生特别贪玩。

分别刻画模糊谓词、模糊修饰词、模糊量词

模糊谓词: 成绩好、刻苦、贪玩

模糊修饰词: 都很、特别

模糊量词: 大多数、很少



prolog

- Prolog是一种基于一阶谓词逻辑的编程语言, 主要用于人工智能和逻辑编程领域。它通过定义事实和规则来进行推理和查询。

决策树 gbrt 逻辑回归

adaboost算法原理

神经网络 CNN ResNet RNN机器翻译

ResNet（残差网络）通过引入“捷径连接”，让网络学习输入与输出之间的残差，从而解决了深度增加带来的训练难题，使超深神经网络变得可行且高效。

rnn机器翻译

- 1.原语种序列送入rnn编码器，得到编码序列
- 2.得到隐藏状态序列
- 3.将隐藏状态送入rnn解码器，初始化解码器，以预测下一词概率的方式生成目标语种序列
- 4.目标误差含数为交叉熵函数，优化目标为最小化交叉熵

囚徒困境（负和）

效用矩阵是元组

传教士和野人

GAN

焚诀

- 选择：
 - 图灵测试
 - 三大学派
 - 多臂赌博机的优化，探索和利用的关系，优化的期望、UCB策略的思想、乐观初值法、马尔可夫决策过程、奖励reward hypothesis假设的概念、策略和状态估值函数
 - 策略的四个概念、策略提升、策略迭代
 - 动态规划 蒙特卡洛 时序差分四个图
 - 田忌赛马、囚徒困境、纳什均衡的概念、零和博弈、囚徒困境是社会最优、帕累托最优
 - 博弈应用案例选择题一个、知道是什么、讨价是双方对商品的价格评估之差
 - maxmin, minmax一道选择题
 - 什么是完全匹配、最优匹配、引导
 - 结局的稳定性、纳什溢价解

选择

1.

这道题考察的是对几种常见搜索算法的理解，我们逐项分析选项：

题目：关于搜索算法，以下说法错误的是（ ）

A. 广度优先搜索是代价一致搜索的一种特殊情况

- 广度优先搜索（BFS）是在无权图中，按层扩展节点的搜索方式。
- 代价一致搜索（Uniform-Cost Search, UCS）是根据路径代价来扩展节点，总是选择当前总代价最小的路径。
- 当所有边的代价都相等时（比如都是1），UCS就退化为BFS。
- 所以，BFS是UCS的一种特殊情况。

✓ 正确。

B. 代价一致搜索是A*搜索的一种特殊情况

- A*搜索使用启发式函数 $h(n)$ 和实际代价 $g(n)$ ，优先级为 $f(n) = g(n) + h(n)$ 。
- 如果启发式函数 $h(n) = 0$ ，那么 $f(n) = g(n)$ ，此时A*就变成了只考虑路径代价的搜索，即代价一致搜索。
- 所以，UCS是A*的特例（当 $h(n) = 0$ ）。（就是dijkstra）

✓ 正确。

C. 贪婪最佳优先搜索是完备的

✗ 错误。

- 贪婪最佳优先搜索（Greedy Best-First Search）只考虑启发式函数 $h(n)$ ，即总是选择看起来“最接近目标”的节点。
- 它不保证找到最优解，也不保证一定能找到解。
- 特别是在存在局部最优或“死胡同”的情况下，它可能陷入循环或无法到达目标。
- 不是完备的（即不能保证在有限时间内找到解，即使解存在）。

D. 爬山法搜索可在任意位置起始，移动到最好的相邻状态

2.

在使用 归结反证法 (resolution refutation) 进行一阶谓词逻辑的自动推理时, 标准流程如下:

✓ 正确步骤顺序 (关键点):

将结论取反 (Negate the conclusion)

→

量词前束 (Prenex normal form)

→

合取范式标准化 (Skolemization + CNF)

→

使用归结原理

3.

◆ Deep Belief Network (DBN)

DBN 的核心结构是: 多个受限玻尔兹曼机 (Restricted Boltzmann Machines, RBMs) 堆叠而成。

✗ 其他选项分析:

A. Boltzmann Machine

→ 普通玻尔兹曼机允许任意连接 (包括隐藏层之间的连接), 计算复杂, 不适合用于堆叠。

→ 不是 DBN 的构建单元。

C. AutoEncoder

→ 自编码器是一种有监督/无监督的压缩模型, 常用于特征学习, 但 不是 DBN 的组成部分。

→ 虽然也有“堆叠自编码器” (Stacked Autoencoder), 但它属于另一种深度网络结构, 与 DBN 不同。

D. Hopfield Networks

→ 是一种循环神经网络, 用于联想记忆, 不用于堆叠构建深度网络。

→ 与 DBN 无关。

4.

A. 局部连接 (Local Connectivity)

CNN 中每个神经元只与输入特征图中的局部区域 (即感受野) 相连。

这模仿了生物视觉系统的局部感知机制。

✓ 是 CNN 的关键特性之一。

✓ B. 参数共享 (Weight Sharing)

卷积核 (filter) 在输入图像上滑动时, 使用相同的权重参数。

这大大减少了模型参数数量, 并增强了平移不变性。

✓ 是 CNN 的核心设计思想。

✓ C. 子采样 (Subsampling)

也叫池化 (Pooling), 如最大池化 (Max Pooling)、平均池化 (Average Pooling)。

用于降低特征图的空间维度, 减少计算量, 增强鲁棒性。

✓ 是 CNN 的标准组成部分。

✗ D. 逐层贪婪训练 (Layer-wise Greedy Training)

这是 深度信念网络 (DBN) 或 受限玻尔兹曼机 (RBM) 的训练方式。

在 DBN 中, 每层 RBM 用无监督方式预训练, 然后逐层堆叠。

而 CNN 通常采用端到端的反向传播训练, 不是“逐层贪婪训练”。

✗ 不属于 CNN 特点。

5.

A. 信息系统在的稳定态某一状态时，该系统的信量最小

✅ A 是正确的表述。

🔍 B. 除了香农熵，信息还有其它可能的定义形式

香农熵 (Shannon Entropy) 是最常见的信息度量方式，但不是唯一的。

其他信息度量包括：负熵 (Negentropy) 相对熵 (KL散度) 条件熵互信息信息距离等在不同领域 (如量子信息、复杂系统) 也有不同的信息定义。

✅ B 是正确的。

C. 决策树算法优先选择使条件熵最大的属性进行样本划分

❌ 这是错误的!

决策树算法 (如 ID3、C4.5) 的目标是 减少不确定性，也就是 最大化信息增益。

信息增益 = 父节点熵 - 条件熵 (子节点的加权平均熵)

所以我们要选择 条件熵最小 的属性，这样信息增益最大。

条件熵越小，说明划分后数据越纯，分类效果越好。

均衡结局

纳什均衡:

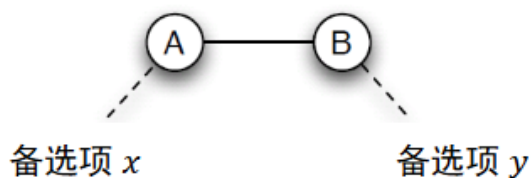
- 每个玩家都选择了最优策略，而不是随机策略或其他策略。

- 每个玩家的策略都是最优的，而不是次优的。

纳什议价解

■ 议价的对象

- 如何分配“剩余价值” $s = 1 - x - y$



■ 纳什议价解

- A的收益是: $x + \frac{s}{2} = \frac{1+x-y}{2}$
- B的收益是: $y + \frac{s}{2} = \frac{1+y-x}{2}$

均衡结局

■ 均衡结局 (balanced outcome)

- 给定一个结局，如果结局中的任意一个参与配对的边都满足纳什议价解的条件，则称该结局是均衡结局

■ 注意：均衡结局一定是稳定结局

- 因此，在寻找均衡结局时，可以先寻找稳定结局，进而确定均衡结局

稳定结局

- 每个玩家的策略都不会改变，无论其他玩家的策略如何。

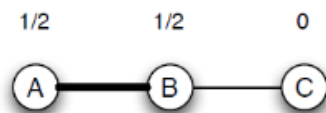
结局的稳定性

■ 不稳定边

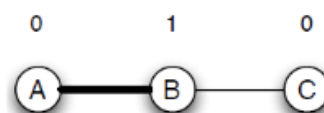
- 对于结局中**未参与配对的边**，如果边的两个端点获得的收益之和小于1，则称这条边为不稳定边
- 不稳定边的存在意味着其两个端点可以通过改变报价而改变结局

■ 稳定结局(stable outcome)

- 如果一个结局中不存在不稳定边，则称该结局为稳定结局



不稳定的结局
B和C的收益之和小于1



稳定的结局

d分离:

- d分离:

因果模型图与独立性

■ d -分离用于确定因果模型图中任意一对节点是否独立

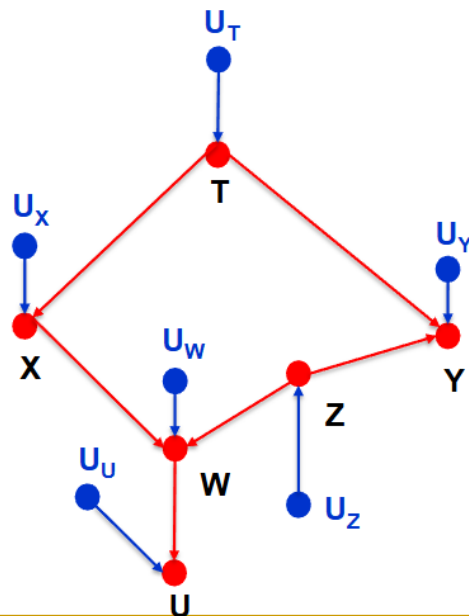
- 一条路径会被一组节点 Z 阻断，当且仅当：
 - 路径 p 包含链结构 $A \rightarrow B \rightarrow C$ 或分叉结构 $A \leftarrow B \rightarrow C$ ，且中间节点 B 在 Z 中；或
 - 路径 p 包含一个对撞结构 $A \rightarrow B \leftarrow C$ ，且对撞节点 B 及其子孙节点都不在 Z 中。
- 如果一组节点 Z 阻断了 X 和 Y 间的每一条路径，则 X 和 Y 在 Z 的条件下是 d -分离的，即 $X \perp Y \mid Z$

103

d - 分离示例

■ 以下一组变量是否能形成变量 X 与 Y 的 d - 分离

- ☐ $\{T\}$ ✓
- ☐ $\{W\}$ ✗
- ☐ $\{T, U\}$ ✗
- ☐ $\{T, Z\}$ ✓
- ☐ $\{W, U, Z\}$ ✗
- ☐ $\{U, Z, T\}$ ✓

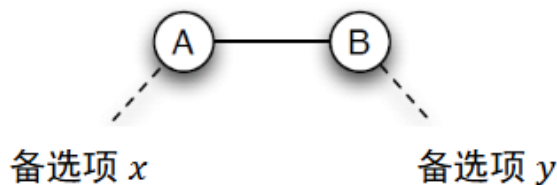


强化学习方法表示

纳什议价解

■ 议价的对象

- 如何分配“剩余价值” $s = 1 - x - y$



■ 纳什议价解

- A的收益是: $x + \frac{s}{2} = \frac{1+x-y}{2}$
- B的收益是: $y + \frac{s}{2} = \frac{1+y-x}{2}$

均衡结局

■ 均衡结局 (balanced outcome)

- 给定一个结局, 如果结局中的任意一个**参与配对的边**都满足纳什议价解的条件, 则称该结局是均衡结局

■ 注意: 均衡结局一定是稳定结局

- 因此, 在寻找均衡结局时, 可以先寻找稳定结局, 进而确定均衡结局

minmax maxmin

- minmax 策略: 最小化对手最好情况下的收益 关注的对抗
- maxmin 策略: 最大化自己最坏情况下的收益 关注的是自己

囚徒困境 (负和) 帕累托最优 社会最优

效用矩阵是元组

- 帕累托最优: 只要没有方案完美diff你, 你就是帕累托最优

社会最优示例

■ 囚徒困境案例

		囚徒B	
		抗拒	坦白
囚徒A	抗拒	-1,-1	-10,0
	坦白	0,-10	-3,-3

帕累托最优的决策组合一共有3个，分别是（坦白，抗拒），（抗拒，坦白）和（抗拒，抗拒），纳什均衡策略组合（坦白，坦白）不是帕累托最优，社会最优策略组合是（抗拒，抗拒）

多臂赌博机

- A. 多臂赌博机的优化目标是每次摇臂的平均期望收益
- B. 贪心策略优化的目标是当前行为的期望收益
- C. UCB 策略随着摇臂次数的增加，最终会收敛到贪心策略
- D. ϵ -贪心策略随着摇臂次数的增加，最终会收敛到贪心策略

//UCB (Upper Confidence Bound) 是一种平衡“探索与利用”的算法，它会给不确定性高的臂更高的优先级。但随着摇臂次数增加，所有臂的不确定性逐渐减小，UCB 的置信区间变窄，最终趋于只选择期望收益最高的臂——即变为贪心策略。

// ϵ -贪心策略中，有一个固定的参数 ϵ (比如 0.1)，表示以概率 ϵ 随机选择一个臂（探索），以概率 $(1-\epsilon)$ 选择当前估计最好的臂（利用）。